

**PROBLEM 5**

# Search Ranking

*Learning to Rank — Ordering results so the most relevant appear at the top*

ML Interview Preparation Guide

## What Is Search Ranking?

You type "red running shoes" into Google. In 0.3 seconds, it shows you 2 billion results in a specific order. The first result is almost always exactly what you wanted. Search ranking = deciding the order in which to show results so the most relevant appear at the top.

Studies show 92% of users never go past page 1. Position 1 gets ~30% of clicks. Position 10 gets ~2%. Getting ranking right is worth billions of dollars.

*Real World Examples: Google/Bing web search, Amazon product search, LinkedIn job search, Airbnb listing search, Spotify song search, Gmail email search.*

## Step 0: Clarify the Problem

- **What are we ranking?** Web pages? Products? Jobs? People?
- **What is the goal?** Clicks? Purchases? Time spent? Different goals = different training signals.
- **What data do we have?** User click logs? Explicit ratings? Purchase history?
- **Personalized or not?** Same query "running shoes" could rank differently for a marathon runner vs. a casual walker.
- **How do we get training labels?** This is the hardest part of search ranking.

## Step 1: Understanding the Data

### The Label Problem

You have three options for getting relevance labels:

- **Human Raters (Explicit Labels):** Pay humans to rate query-document pairs on a 0-4 scale. High quality but expensive and slow.
- **Click Data (Implicit Labels):** Use user behavior — clicks, purchases, long dwell time, pogo-sticking (quick back-click). Free and abundant but noisy and biased.
- **Hybrid:** Use human labels to calibrate your click-based model.

### Position Bias — The Critical Problem

Users click on top results partly because they are at the top, not just because they are relevant. Position 1: 35% CTR. Position 5: 6% CTR. Position 10: 2% CTR. If you train naively on clicks, your model learns position 1 things are relevant and keeps ranking them first — a self-fulfilling prophecy.

Fix: Inverse Propensity Scoring (IPS). Weight each click by the inverse of how likely that position is to be clicked. A click from position 5 counts much more than from position 1.

## Step 2: The Search Pipeline

Search ranking is ALWAYS a multi-stage pipeline:

1. QUERY UNDERSTANDING: Parse intent, spell correction ("runing shoez" to "running shoes"), query expansion (add synonyms).
2. RETRIEVAL: From millions of documents, find ~1000 candidates using inverted index. Must be extremely fast.
3. RANKING (Learning to Rank): Score and rank the 1000 candidates using ML model with rich features.
4. RE-RANKING / BUSINESS RULES: Apply diversity, freshness, inject ads, apply business constraints.

## Step 3: Feature Engineering

### Query Features

- `query_length`, `query_type`, `is_brand_query`, `query_frequency`
- `query_has_location` — "pizza near me"
- `user_search_history` — has user searched similar things before?

### Document Features

- `total_sales_30d`, `page_views_7d`, `avg_rating`, `num_reviews`
- `return_rate` — high returns = bad product, should rank lower
- `is_prime_eligible`, `seller_reputation_score`, `is_in_stock`

### Query-Document Relevance Features (Most Important)

- `query_title_exact_match` — does query appear exactly in title?
- `query_title_word_overlap` — what fraction of query words appear in title?
- `query_document_cosine_similarity` — semantic (meaning-level) match using embeddings
- `ctr_for_this_query_doc_pair` — have users clicked this product for THIS query before?

### TF-IDF — The Classic Text Matching Feature

TF (Term Frequency): how often does the query word appear in the document? IDF (Inverse Document Frequency): how rare is this word across all documents? Rare, specific words carry more signal.

TF-IDF = TF x IDF. A document containing rare, specific words from your query is more relevant than one containing only common words.

## BM25 — The Gold Standard Retrieval Score

BM25 improves TF-IDF with two key ideas: (1) TF saturation — mentioning a word 10 times is not 10x better than once; BM25 caps this effect. (2) Length normalization — a long document mentioning a word 5 times is not necessarily more relevant than a short document mentioning it 3 times. BM25 normalizes for document length.

## Step 4: Statistics and Math

### NDCG — The Core Evaluation Metric

NDCG (Normalized Discounted Cumulative Gain) cares about ORDER. Showing a great result at position 1 is worth more than position 10.

- Assign relevance grades: purchased = 3, clicked = 2, skipped = 0
- $DCG = \text{sum of } (\text{relevance}_i / \log_2(\text{position}_i + 1))$
- $IDCG = \text{DCG of the ideal (perfect) ranking}$
- $NDCG = DCG / IDCG$  (1.0 = perfect, 0.0 = worst)

Lower positions get less credit (discounted). NDCG is the primary metric for all search ranking systems.

## Step 5: Models — Learning to Rank

### Approach 1: Pointwise Learning to Rank

Treat ranking as regression on individual documents. For each query-document pair, predict a relevance score. Train any regression model (LightGBM, neural network). Rank by predicted score. Simple but ignores the fact that ranking is about relative order.

### Approach 2: Pairwise Learning to Rank

For each query, take pairs of documents and predict which is more relevant. If user clicked Doc\_A but not Doc\_B for the same query, Doc\_A is more relevant. RankNet (Microsoft, 2005) is the classic pairwise model.

$P(A \text{ ranked above } B) = \text{sigmoid}(\text{Score}_A - \text{Score}_B)$ . The model learns to give higher scores to more relevant documents. Pros: more natural for ranking. Cons: quadratic number of pairs, all pairs weighted equally.

### Approach 3: Listwise Learning to Rank — LambdaMART

Optimize the entire ranked list at once — directly maximize NDCG. Industry standard, used by Microsoft Bing, Yahoo, and many others.

NDCG is not differentiable (cannot take its gradient), so LambdaRank defines a virtual gradient (lambda) that approximates how much swapping two documents would improve NDCG.

LambdaMART = LambdaRank + gradient boosted trees.

- **Pros:** Directly maximizes NDCG, best accuracy in practice, fast gradient boosted trees
- **Cons:** More complex to implement than pointwise/pairwise

## Neural Ranking Models

### DSSM (Deep Structured Semantic Model)

Two separate neural networks for query and document, compute cosine similarity between their output vectors. Uses character n-grams to handle spelling variations (runing and running share most character 3-grams). Captures semantic meaning beyond keyword matching.

### BERT Cross-Encoder

Feed query AND document together into BERT: [CLS] red running shoes [SEP] Nike Air Zoom Red Marathon Shoe [SEP]. BERT attends across query and document — notices that "running" matches "marathon" semantically. Most accurate but very slow — only used in reranking stage on top 100 candidates.

### ColBERT — The Middle Ground

Score = sum over query tokens of max similarity to any document token. More accurate than two-tower (tokens interact). Faster than cross-encoder (document embeddings precomputed offline). Storage-heavy (stores all token embeddings for all documents).

## The Full Modern Ranking Stack

Query goes through: (1) BM25 via Elasticsearch retrieves 500 keyword-match candidates PLUS (2) Two-Tower ANN search retrieves 500 semantic-match candidates. Merged to ~800 unique candidates. Then (3) LambdaMART ranks to top 50. Then (4) BERT cross-encoder reranks to top 20. Finally (5) business rules apply: inject ads, ensure diversity, boost in-stock items.

## Step 6: Evaluation

### Offline Metrics

- **NDCG@K:** Primary metric. Measures quality of top K results with position discounting.
- **MRR (Mean Reciprocal Rank):**  $1 / \text{rank of first relevant result}$ . Great for navigational queries.
- **MAP (Mean Average Precision):** Average precision across all relevant results.

### Online Metrics

- **CTR:** What % of search results get clicked?
- **Conversion rate:** What % lead to a purchase?
- **Zero-result rate:** How often does search return nothing? (Bad!)
- **Search abandonment:** User searched but did not click anything — relevance failure.
- **Reformulation rate:** User immediately searched again — first results were bad.

## Step 7: Production

### Query Understanding Pipeline

- **Spell correction:** "runing shoez" to "running shoes" using noisy channel model or Transformer
- **Query expansion:** "running shoes" also searches "sneakers", "trainers" using Word2Vec synonyms
- **Intent classification:** Navigational (want specific page) vs Informational (want article) vs Transactional (want to buy)
- **Entity recognition:** "red nike pegasus size 10" parsed into brand, product, color, size as structured filters

### Handling Long-Tail Queries

Head queries (popular) have lots of click data — ML works great. Tail queries (rare) have almost no click data — model falls back to BM25 + content features. Solutions: query clustering (share data across similar queries), zero-shot ranking with BERT embeddings that generalize, query rewriting with LLMs.

### Freshness vs. Relevance Tradeoff

For queries like "Best laptops 2024", a 2019 article is less relevant despite good SEO. Freshness score =  $\exp(-\lambda \times \text{days\_since\_published})$ . Blend with relevance:  $\text{final\_score} = \alpha \times \text{relevance} + (1-\alpha) \times \text{freshness}$ . Alpha is higher for evergreen queries, lower for news/trends queries.

### Monitoring

- Zero-result rate per query category
- NDCG on held-out query set — daily
- Feature distribution drift
- Latency percentiles: P50, P95, P99
- Revenue per search and search-to-purchase conversion rate

## How to Answer in an Interview

*Search ranking is a multi-stage problem. Clarify what we are optimizing for — clicks, purchases, or engagement — and whether we need personalization. The pipeline: retrieval (BM25 via Elasticsearch plus semantic two-tower search for 1000 candidates), ranking (LambdaMART — listwise LTR that directly optimizes NDCG using gradient boosted trees), reranking (BERT cross-encoder on top 50 for maximum accuracy). Features: query features like intent and length, document features like ratings and sales, and crucially query-document matching features. For labels: use click data with inverse propensity scoring to correct position bias. Evaluate with NDCG@10 offline and A/B test revenue per search online. Key production concerns: query understanding pipeline, feature store for low-latency serving, and semantic embeddings for long-tail queries.*